

DOCUMENT EXTRACTION PILOT REPORT

Evaluation of a Function-Calling Workflow for Structured PDF Analysis

Prepared by: Applied AI Engineering Team
Date: August 3, 2026
Report ID: AAI-REP-2026-08

EXECUTIVE SUMMARY

This report evaluates a prototype that extracts text from PDF files, classifies each document, and routes supported document types to specialized Python tools. The pilot focused on invoices, resumes, receipts, and reports. The workflow successfully demonstrated local tool execution, structured validation, and multi-step orchestration using a Groq-hosted language model.

OBJECTIVES

- Verify reliable text extraction from text-based PDFs.
- Measure document classification accuracy across four document types.
- Validate specialized field extraction for invoices and resumes.
- Identify operational risks before expanding to production use.

METHODOLOGY

The test set contained 40 synthetic, text-based PDF documents: 10 invoices, 10 resumes, 10 receipts, and 10 analytical reports. Each PDF was processed with pypdf. The extracted text was sent to an LLM that first called a document-classification tool. When the result was an invoice or resume, the model called a specialized extraction tool. Tool outputs were validated with Pydantic schemas.

FINDINGS

Metric	Result	Interpretation
PDF text extraction success	40 / 40 (100%)	All test PDFs contained embedded text.
Document classification accuracy	38 / 40 (95%)	Two receipts were incorrectly labeled as generic.
Invoice field completeness	93%	Most missing values involved optional due dates.
Resume field completeness	90%	Experience entries varied in formatting.
Average processing time	2.8 seconds	Suitable for interactive testing.

RISKS AND LIMITATIONS

- Scanned PDFs require OCR and are outside the current implementation.
- Long documents can exceed the model context window without chunking or compression.
- The model may omit fields, choose an incorrect tool, or generate invalid arguments.
- Personally identifiable information should be minimized, protected, and logged carefully.

RECOMMENDATIONS

1. Add a dedicated receipt extraction schema and tool.
2. Return a validated top-level JSON response instead of model-formatted Markdown.
3. Add retry handling for invalid tool arguments and maximum-tool-round protection.
4. Build a golden evaluation dataset with expected classifications and extracted fields.
5. Add OCR only after the text-based workflow is stable.

CONCLUSION

The pilot confirms that a raw function-calling architecture can classify text-based PDFs and extract structured fields through local Python tools. The prototype is ready for expanded automated testing, top-level JSON validation, and additional specialized document schemas.

APPENDIX A - TEST DOCUMENT TYPES

Invoice: billing document with vendor, customer, dates, line items, subtotal, tax, total, and currency.

Resume: candidate profile with contact information, summary, skills, education, and experience.

Receipt: point-of-sale record with merchant, timestamp, purchased items, subtotal, sales tax, payment method, and amount paid.

Report: formal analytical document with executive summary, methodology, findings, limitations, recommendations, and conclusion.